

Project Report

Students

Amirmohammad Kooshky - 400109673

Mohsen Ghasemi - 400105166

Nima Ghadirniya - 402111323

Task Allocation

Design of Entities:

all members participated to draw out the entities of the project.

Design of Django Apps:

all members participated to draw out the design

Implementation of Initial Django Models:

Kooshky and Ghasemi (separated by apps)

Creation of scripts that build context for LLMs:

Kooshky and Ghasemi: the scripts coalesced the project's code files into the same file that could be fed into an LLM.

Implementation of Serializers and Views:

Kooshky and Ghasemi (separated by apps)

Testing of API's using curl and POSTMAN:

Ghadirniya (so he is onboarded to the backend and can connect to front)

Testing of API's using automated library (Unit Tests):

Ghadirniya and Ghasemi

Frontend Architecture and Route Structure:

Ghadirniya

Frontend UI/UX Design System (theme, layout consistency, component styling):

Ghadirniya

Implementation of Frontend Pages and Role-Specific Dashboards:

Ghadirniya

Frontend API Integration and Backend Synchronization:

Ghadirniya

Frontend Workflow Implementation (forms, validation flows, user actions):

Ghadirniya

Frontend Error Handling, Loading States, and Skeleton Layouts:

Ghadirniya

Testing of the Frontend:

Kooshky and Ghasemi (to understand how the frontend is written)

Connecting to Payment API:

Assigned to Ghasemi (if we had enough time)

Development Agreements:

Commits and Git

At first commits were just required to be clean and descriptive. later we switched to conventional commits for the frontend (Ghadirniya)

Merge requests were peer reviewed (especially on the backend)

Variable Naming

The project follows standard Python and Django naming conventions, and these rules were explicitly defined and provided to the LLM during development to ensure generated code remained consistent with the existing codebase.

For **Python variables, functions, and methods**, `snake_case` naming is used throughout the project. Variable names are descriptive and reflect domain concepts rather than implementation details (e.g., `assigned_detective`, `arrest_status`, `crime_level_weight`). Temporary or computed values are named according to their semantic meaning, such as `tracking_started_at`, `ranking_score`, and `reward_amount_rial`.

For **classes and Django models**, `PascalCase` is used, following Django conventions. Model names represent real-world entities in the crime management domain, such as `Case`, `CaseSuspect`, `Complainant`, `RewardTip`, and `InvestigationAction`. View and serializer classes clearly indicate their responsibility using suffixes like `View`, `ListView`, `DetailView`, `Serializer`, or `ViewSet` (e.g., `CaseReportView`, `CaseSuspectCreateUpdateView`).

Constants and role identifiers are written in uppercase with underscores and centralized in dedicated modules to avoid magic strings and improve maintainability. Examples include `DETECTIVE`, `POLICE_CHIEF`, `CAPTAIN`, and status values such as `OPEN`, `UNDER_INVESTIGATION`, and `ARRESTED`.

For **Django relationships**, field names and `related_name` values are chosen to be explicit and readable when traversing relations in queries and serializers. For example, `assigned_detective`, `sergeant_cases`, `detective_cases`, and `testimonies` clearly describe the direction and meaning of each relationship. Plural forms are consistently used for reverse relations to reflect collections.

API-related variables and serializer fields mirror model field names whenever possible, ensuring a predictable mapping between database models, serializers, and JSON responses. This reduces cognitive overhead for both frontend integration and future maintenance.

By defining and enforcing these naming conventions early, and by feeding them as explicit constraints to the LLM during code generation, the team ensured that AI-assisted code blended seamlessly with manually written code and adhered to Django and Python best practices across the entire project.

The frontend follows standard **JavaScript/React naming conventions**, and these rules were also explicitly defined and provided to the LLM during development so that generated code stayed consistent with the project's existing React/Vite codebase.

For **JavaScript variables, functions, and methods**, `camelCase` is used throughout the frontend. Names are chosen based on domain meaning and UI behavior rather than low-level implementation details (e.g., `selectedCaseId`, `readOnlyBoard`, `canReadEvidence`, `loadReport`, `createBoardRelation`, `formatUiApiError`). Boolean variables use clear prefixes such as `is`, `has`, and `can` (e.g., `isLoading`, `hasAccess`, `canEditLayout`) to make conditional logic easy to understand.

For **React components, pages, and context providers**, `PascalCase` is used consistently, following React conventions. Component names clearly reflect responsibility and screen/module boundaries, such as `CaseDetailPage`, `DetectiveBoardPage`, `TipSubmitPage`, `BailPaymentReturnPage`, `AuthContext`, and `RoleProtectedRoute`. Reusable UI building blocks and dialogs also use descriptive suffixes such as `Page`, `Modal`, `Dialog`, `Provider`, and `Route`.

For **React state and setters**, the project follows the standard `useState` naming pattern (`[value, setValue]`), which improves readability and consistency in stateful components (e.g., `loading / setLoading`, `board / setBoard`, `error / setError`). Event handlers and async actions are named with verbs to reflect intent, such as `handleSubmit`, `loadAll`, `deleteRelation`, and `saveDetectiveBoardLayout`.

Constants and configuration values are written in uppercase with underscores and grouped in dedicated modules or top-level constant sections to avoid magic values and simplify maintenance. Examples include `API_BASE_URL`, `ADMIN_QUEUE_TYPES`, and `EVIDENCE_TYPES`. This convention is also used for cache keys and feature flags where applicable.

For **API integration and serializer-compatible payloads**, the frontend intentionally preserves backend field names (typically `snake_case`) in request/response objects whenever possible, especially inside `api.js` and normalization helpers. Examples include fields such as `assigned_detective`, `arrest_status`, `judicial_outcome`, and `bail_paid_at`. At the same time, frontend-only state and component logic continue to use `camelCase`, creating a clear distinction between UI state and backend contracts while keeping the mapping predictable.

For **utility and normalization functions**, names are descriptive and transformation-oriented, clearly indicating their role in adapting backend data for the UI layer. Examples include `normalizeCaseEntity`, `normalizeSuspectEntity`, `mapCrimeLevelToLevel`, and `resolvePublicAssetUrl`. This naming pattern makes data flow between backend responses and frontend components easier to maintain and debug.

By defining and enforcing these frontend naming conventions early, and by giving them as explicit constraints to the LLM during code generation, the team ensured that AI-assisted frontend code remained consistent with React best practices, aligned with backend API contracts, and integrated cleanly with manually written components and modules across the project.

Project Management: Task Generation and Distribution

Project management for this project followed a lightweight and practical approach, suitable for a small development team and a short timeline. Instead of rigid phase-based milestones, tasks were continuously generated, refined, and redistributed as the system design evolved.

Task Generation

Tasks were primarily derived from:

- The official project description and functional requirements
- Early domain analysis and entity design sessions
- Incremental discoveries during implementation (e.g., missing models, permission gaps, API inconsistencies)
- Integration needs between backend and frontend components

Initial tasks focused on establishing the core domain models and Django app structure. As development progressed, tasks naturally expanded to include serializer design, API endpoint implementation, permission handling, testing, and frontend integration. Bug fixes and refactors were treated as first-class tasks rather than afterthoughts.

To support AI-assisted development, custom scripts were created to automatically gather and merge relevant parts of the codebase into a single contextual input. This allowed new implementation tasks to be generated or refined using LLMs while preserving architectural and naming consistency across the project.

Task Distribution

Task distribution was based on **area ownership rather than rigid roles**. Team members were assigned responsibilities aligned with their familiarity with specific Django apps or system layers (e.g., accounts, cases, investigations, frontend). This minimized context-switching and improved development speed.

Core backend responsibilities, such as model implementation, serializers, views, and permission logic, were primarily handled by Kooshky and Ghasemi, with tasks divided by Django app. This separation allowed parallel development while maintaining clear ownership boundaries. Ghadirniya was responsible for the testing and automated tests for the backend.

System Key Entities and Their Design Rationale

During the analysis and design phase, the system was decomposed into a set of persistent entities based on software engineering principles such as separation of concerns, data normalization, lifecycle independence, and future extensibility. Each entity was introduced to encapsulate a stable concept with its own state, behavior boundaries, and persistence requirements.

User

The **User** entity was defined as a foundational abstraction to unify authentication, authorization, and identity management across the system. From a backend perspective, representing all

actors as a single user entity avoids duplication of identity data and enables centralized access control, auditing, and relationship management. Role distinctions are treated as attributes or associations rather than separate entities to preserve flexibility and reduce schema complexity.

Case

The **Case** entity was introduced as the system's primary aggregate root. From a data modeling standpoint, it provides a stable parent context that groups all related information under a single transactional and conceptual boundary. This allows other entities to evolve independently while remaining consistently associated with a case, and enables efficient querying, access control enforcement, and lifecycle management at the case level.

Evidence Entities

Evidence was modeled as multiple specialized entities rather than a single polymorphic structure to maintain clarity, validation correctness, and domain-specific constraints at the data layer. Each evidence type has a distinct lifecycle, validation logic, and review process, which justifies separate persistence models.

- **Testimony Evidence** exists to represent unstructured human-provided information. From a backend design perspective, it requires different validation rules and storage considerations compared to physical or forensic evidence.
- **Biological Evidence** was separated due to its need for expert review states and traceable verification history. This entity encapsulates state transitions and review metadata that would otherwise complicate a generic evidence model.
- **Vehicle Evidence** was introduced to enforce structural constraints specific to vehicles (such as mutually exclusive identifiers) at the database level, improving data integrity and reducing reliance on application-layer validation.
- **Identification Document** exists to model semi-structured identity artifacts whose attributes may vary across document types. Treating it as a separate entity allows flexible storage while preserving a consistent association with a case.
- **Other Evidence** was deliberately added as an extensibility mechanism. From a software design perspective, it prevents frequent schema changes when new or rare evidence types arise, supporting open–closed design principles.

Reward Tip

The **Reward Tip** entity was designed to represent externally submitted information with an independent lifecycle and uncertain reliability. From a backend standpoint, separating tips from formal evidence allows independent state transitions, review workflows, and auditability. This separation also reduces coupling between user-submitted content and verified investigative data.

Trial

The **Trial** entity was introduced to represent a distinct processing phase with different authority, access rules, and data visibility requirements. From a system design perspective, separating trials from cases avoids overloading the case entity with judicial state and enables clear boundaries between investigative and judicial workflows.

Punishment

The **Punishment** entity was intentionally separated from the trial to reflect conditional existence and delayed creation. This design avoids nullable or overloaded fields within the trial entity and allows punishment data to evolve independently, adhering to normalization and single-responsibility principles.

Notification

The **Notification** entity was added as an infrastructure-level concern rather than a domain artifact. From a backend perspective, it decouples event generation from message delivery, enabling asynchronous communication, persistence of unread state, and future extensibility toward real-time or external notification mechanisms without impacting core domain entities.

Overall, these entities were identified and justified based on backend stability, clear responsibility boundaries, and long-term maintainability. This design-first approach ensured that the persistence layer accurately reflected system responsibilities before any implementation details were introduced.

NPM packages

The frontend intentionally uses a small set of NPM packages to keep the stack maintainable while still covering routing, styling, and build tooling needs. The following packages (maximum six) are the primary ones used in the project, along with a short summary of their role and the justification for using them.

1. react

React is the core UI library used to build the frontend as a component-based single-page application. It enables modular page and component design (e.g., dashboards, case pages, detective board, and admin panels) and supports predictable state-driven rendering across the system.

Justification: The project contains many role-based screens and reusable UI sections. React provides a clean component model that reduces duplication and makes complex UI flows easier

to maintain.

2. `react-dom`

`react-dom` is used to render React components into the browser DOM and provides the runtime integration between the React component tree and the HTML page.

Justification: Since the frontend is implemented as a browser-based React application, `react-dom` is the standard and necessary package for mounting and updating the UI.

3. `react-router-dom`

`react-router-dom` is used for client-side routing and navigation between pages such as login, dashboard, cases, reports, detective board, bail/payment return page, and admin modules. It also supports protected routes and role-based navigation flows.

Justification: The project contains many role-specific pages and workflows, so declarative routing is essential for maintainability and for enforcing navigation structure without full page reloads.

4. `vite`

`Vite` is used as the frontend development server and build tool. It provides fast startup, hot module replacement during development, and optimized production builds.

Justification: Vite significantly improves developer productivity compared to heavier bundlers, especially in a project with frequent frontend iteration, UI adjustments, and backend integration testing.

5. `tailwindcss`

Tailwind CSS is used as the primary styling framework for the frontend. The project defines a custom theme (colors, fonts, shadows, and utility extensions) and uses utility classes to build consistent layouts and components across pages.

Justification: The UI includes many pages and role-specific panels. Tailwind made it possible to develop and refine the interface quickly while preserving visual consistency and reducing custom CSS overhead.

6. `@vitejs/plugin-react`

This Vite plugin enables React-specific support in the build pipeline, including JSX transformation and development optimizations required for a smooth React development workflow.

Justification: Since the frontend stack is based on React + Vite, this plugin is the standard integration layer that ensures the React codebase works correctly with Vite in both development and production builds.

AI-Generated Code Samples:

```
class Notification(models.Model):

    class NotifType(models.TextChoices):
        TIP_FORWARDED = 'tip_forwarded', 'Tip Forwarded to Detective'
        TIP_CONFIRMED = 'tip_confirmed', 'Tip Confirmed – Reward Issued'
        TIP_REJECTED = 'tip_rejected', 'Tip Rejected'
        BOUNTY_PAID = 'bounty_paid', 'Bounty Paid'
        BOUNTY_REVOKED = 'bounty_revoked', 'Bounty Revoked'
        CASE_UPDATED = 'case_updated', 'Case Updated'
        TRIAL_VERDICT = 'trial_verdict', 'Trial Verdict Recorded'
        GENERAL = 'general', 'General'

    recipient = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='notifications'
    )
    notif_type = models.CharField(max_length=30, choices=NotifType.choices,
default=NotifType.GENERAL)
    title = models.CharField(max_length=255)
    body = models.TextField()
    is_read = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    read_at = models.DateTimeField(null=True, blank=True)

    # Optional generic link to the object that triggered the notification
    link = models.CharField(max_length=500, blank=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return f"[{self.notif_type}] → {self.recipient} ({'read' if
self.is_read else 'unread'})"
```

```
# get_tokens.py
```

```
from django.contrib.auth import get_user_model
from rest_framework.authtoken.models import Token
```

```

User = get_user_model()

print("-" * 60)
print("Fetching tokens for all users...")
print("-" * 60)

users = User.objects.prefetch_related('groups').order_by('username')

for user in users:
    token, _ = Token.objects.get_or_create(user=user)
    roles = ", ".join(user.groups.values_list('name', flat=True)) or "no role"
    print(f"{user.username:<25} {token.key} ({roles})")

print("-" * 60)
print(f"Total: {users.count()} users")
print("-" * 60)

```

```

# cases/admin.py
from django.contrib import admin
from .models import Case, Complainant, CaseWitness, CaseSuspect

class ComplainantInline(admin.TabularInline):
    model = Complainant
    extra = 0
    raw_id_fields = ('user',)

class CaseWitnessInline(admin.TabularInline):
    model = CaseWitness
    extra = 0

class CaseSuspectInline(admin.TabularInline):
    model = CaseSuspect
    extra = 0
    raw_id_fields = ('suspect',)
    readonly_fields = ('arrest_warrant_issued_at',)

@admin.register(Case)
class CaseAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'crime_level', 'status',

```

```

'creation_method', 'assigned_detective', 'assigned_sergeant', 'created_at')
    list_filter = ('crime_level', 'status', 'creation_method')
    search_fields = ('title', 'description', 'location')
    raw_id_fields = ('registered_by', 'assigned_detective',
'assigned_sergeant')
    readonly_fields = ('created_at', 'updated_at')
    inlines = [ComplainantInline, CaseWitnessInline, CaseSuspectInline]
    fieldsets = (
        ('Basic Info', {'fields': ('title', 'description', 'crime_level',
'status', 'creation_method')})),
        ('Location & Time', {'fields': ('location', 'incident_datetime')})),
        ('Assignment', {'fields': ('registered_by', 'assigned_detective',
'assigned_sergeant')})),
        ('Timestamps', {'fields': ('created_at', 'updated_at')})),
    )

@admin.register(CaseSuspect)
class CaseSuspectAdmin(admin.ModelAdmin):
    list_display = ('id', 'suspect', 'case', 'arrest_status',
'detective_guilt_score', 'sergeant_guilt_score')
    list_filter = ('arrest_status',)
    search_fields = ('suspect__username', 'suspect__national_id')
    raw_id_fields = ('suspect', 'case')
    readonly_fields = ('arrest_warrant_issued_at',)

```

Strengths and Weaknesses of AI in Frontend Development

These are our experiences from using AI for frontend development.

Strengths

One major strength of AI is rapid code generation for common UI patterns. Tools such as **GitHub Copilot** and **ChatGPT** are effective at producing boilerplate React components, form handling logic, and CSS layouts, which reduces repetitive work. AI is also helpful for refactoring, explaining unfamiliar frontend code, and suggesting accessibility improvements (e.g., ARIA attributes).

Weaknesses

AI struggles with complex state management, performance-critical rendering paths, and large, evolving frontend codebases. Generated code often lacks architectural consistency and may not align with project-specific conventions. Additionally, AI tools frequently produce visually correct but semantically flawed UIs, particularly in accessibility and edge-case handling.

Another limitation is overconfidence: AI may generate code that appears correct but fails under real user interaction, requiring careful human review.

Comparison of Common AI Tools

- **ChatGPT:** Strong at explaining frontend concepts, generating small components, and reasoning about UI behavior; weaker at maintaining long-term project context.
- **GitHub Copilot:** Excellent inline autocomplete and pattern completion; limited understanding beyond local file context.
- **Claude:** Best AI we worked with if given the right context and if it is not overwhelmed with context and logic.

Overall, AI is best used as an assistive tool in frontend development, with human developers remaining responsible for architectural decisions, usability, and long-term maintainability.

Strengths and Weaknesses of AI in Backend Development

Strengths

AI is well-suited for generating boilerplate backend code such as CRUD endpoints, serializers, authentication flows, and configuration files. Tools like **ChatGPT** and **GitHub Copilot** can quickly scaffold Django or REST API components, helping developers move faster during early development. AI is also useful for explaining already-written code (e.g. from another team mate), suggesting refactors, and drafting unit tests for existing logic.

Another strength is cross-layer reasoning: AI can relate models, APIs, and business logic, making it helpful when designing endpoints or reviewing request–response flows.

Weaknesses

AI struggles with system-level backend concerns such as data consistency, transactional boundaries, performance optimization, and security-critical logic. Generated code may overlook edge cases, produce inefficient queries, or introduce subtle vulnerabilities if used without review. Additionally, AI has limited awareness of project-specific constraints and often fails to maintain architectural consistency across a large codebase.

Comparison of Common AI Tools

- **ChatGPT:** Strong at backend reasoning, architecture discussion, and explaining complex logic; weaker at maintaining full project context over time.
- **GitHub Copilot:** Excellent for inline code completion and repetitive patterns; limited understanding beyond the immediate file.

- **Specialized Dev Tools (e.g., Cursor, Codeium):** Helpful for navigating and editing larger codebases; still require human judgment for design and security decisions.

Evolution of Requirement Assessments:

Models:

A lot of the decisions made about model fields later proved inadequate. They failed to support the intricate state transitions sanctioned by the project description. For example, the workflow of a Case Request becoming a legitimate case or being dismissed involves complex state transitions warranting detailed access control and authorization. We initially only provisioned a state variable for these and hoped to be able to implement the logic using branching in serializers and views. But this severely hindered our ability to keep our APIs restful and clean and threatened to diverge us to an RPC schema for APIs. So we had to extensively refactor and change the models to better reflect the intricate details.

Also, development of the frontend was parallelized with the backend and forced us to use mock API results in its development. The mismatch between mock APIs and real ones implemented in the backend created a lot of complications when connecting the two. It forced us to do refactoring in both the frontend and the backend to bridge the semantic gaps.

Our decisions about app separation and the entities themselves underwent very little change throughout development and turned out to be very robust and adequate. We believe the decisions involved there turned out to be one of the strong points of our ad-hoc design and development paradigm.